Digitization Guide: SJCET Weather Register →

`sjcet_daily.csv`

Owner: Digitization track (1 member) · **Output contract:** `data-contracts.md` §1–3

Follow the steps in order. Each step ends with a **Check**. Don't move on until it passes. Keep notes of anything odd in `digitization/NOTES.md`; it becomes the "challenges" section of the report.

What you are working with

Source	Where	What
Phone photos	`About/Dataset/Main/*.jpg`	289 photos, 4000×2252, **rotated 90°**, some at an angle
Clean scans	`About/Dataset/Main/July2010-Dec2016.pdf`	79 pages, **in reverse order** (latest first)
Monthly summaries	`About/Dataset/Sum/July2010_Dec2016.pdf`	monthly + year-to-date rainfall totals, used as **checksums**

Register columns (the **order changes between years**): Date, Time (~8:30–9:00 AM), Rainfall (**cm**), Dry bulb, Wet bulb, Min, Max, Remarks. There are no humidity or wind columns; humidity is calculated later from dry/wet bulb. Weekends and holidays are blank.

Already built (in `digitization/`):

- `glosat/`: CPU Docker image of the GloSAT table-structure model (`build.sh`, `run.sh`, ~5 s/page)
- `grid_cells.py`: fallback cell cropper using ruled lines
- `value_crops.py`: crops handwritten values directly (no table structure needed)
- `clean_cell.py`: removes ruled lines and tightly crops a cell for OCR
- `ocr_test.py` + `eval/sample_cells/labels.csv`: 150 labelled cells and an accuracy test

Current status: zero-shot TrOCR gets only **~33% exact match**, and dates ~0%. The big jobs are steps 6–7 (labelling + fine-tuning OCR) and step 10 (human review).

Step 0: Setup (½ day)

```
git clone <repo> && cd TransferLearningBasedWeatherForecasting
git checkout -b digitization

# Python env (CPU)
cd digitization && uv sync && cd ..

# GloSAT table model (needs Docker Desktop with WSL integration on Windows)
git clone https://github.com/stuartemiddleton/glosat_table_dataset ../glosat_table_dataset
#   download models.zip from Zenodo record 5363457, unzip into ../glosat_table_dataset/models/
digitization/glosat/build.sh                # builds image "glosat-cpu" (slow, once)
```

Also install `poppler-utils` (needed by `pdf2image`): `sudo apt install poppler-utils`.

Check: `cd digitization && uv run python ocr_test.py --models small` prints accuracies (~30%), and `glosat/run.sh` on one test image writes an XML and an annotated image.

Step 1: Inventory every page → `pages.csv` (1–2 days, manual)

This is the most important manual step. Every later step depends on knowing **which month/year each page is and which column order it uses**.

Create `digitization/work/pages.csv` (a spreadsheet is fine, export to CSV):

column	example	notes
`page_id`	`p0001`	your own stable id, never reuse
`source`	`Main/20260908_101323.jpg` or `Main/July2010-Dec2016.pdf#12`	file, plus PDF page number
`year`, `month`	`2012`, `7`	read from the page header
`half`	`1`	if a month spans two pages: `1`, `2`; else empty
`layout_id`	`L2`	see step 3
`duplicate_of`	`p0102`	if the same month appears as both photo and scan
`quality`	`good` / `blurry` / `torn` / `skewed`	
`notes`	`ink faded bottom half`	

Tips:

- Make a quick contact sheet to go fast: `uv run python -c "..."` or just open the folder in an image viewer at thumbnail size.
- **Duplicates:** if a month exists as both a clean scan and a photo, use the **scan** and mark the photo `duplicate_of`. Keep both rows.
- List months that are **missing entirely**; that's a finding for the report.

Check: every image and every PDF page has exactly one row. A pivot of year × month shows the coverage, and you can state "we have N months from YYYY to YYYY, M missing".

Step 2: Prepare page images → `work/pages/{page_id}.png` (1 day)

Write `digitization/prep_pages.py` that, for each row of `pages.csv`:

1. **PDF pages:** render with `pdf2image.convert_from_path(pdf, dpi=300, first_page=n, last_page=n)`.
2. **Photos:** rotate 90° (check the direction on a few), then **perspective-correct**: find the largest 4-corner contour (the page), `cv2.getPerspectiveTransform` → `cv2.warpPerspective`. If corner detection fails, fall back to rotate + deskew only and set `quality=skewed`.
3. **Deskew** small angles using the ruled horizontal lines (`cv2.HoughLinesP`, take the median angle, rotate by it).
4. Convert to greyscale, apply mild contrast normalisation (CLAHE). Don't binarise here; the OCR model prefers greyscale.
5. Resize so the page is ~2500 px tall (similar scale for scans and photos).

Check: open 20 random outputs. Ruled lines are horizontal, the page fills the image, and text is upright. Fix or flag the rest.

Step 3: Define column layouts → `layouts.csv` (½ day)

Look at one page per year and write each distinct column order once:

```
layout_id,columns
L1,date|time|rain|dry|wet|min|max|remarks
L2,date|time|dry|wet|max|min|rain|remarks
```

Fill `layout_id` for every page in `pages.csv`. Also note whether **row 1 = day 1** or whether there are header rows, and how many (`header_rows` column in `layouts.csv`).

Check: every page has a `layout_id`. Spot-check 5 pages per layout.

Step 4: Table structure, find the cells (2 days)

Primary: **GloSAT** (fine-cell mode):

```
digitization/glosat/run.sh digitization/work/pages digitization/work/glosat fine
```

This gives an XML per page with cell boxes, plus an annotated image.

Fallback for pages where GloSAT fails: `grid_cells.py` (uses the ruled lines):

```
cd digitization && uv run python grid_cells.py work/pages/p0001.png work/grid/p0001/
```

Write `digitization/assign_cells.py` that turns the cell boxes into a **row × column grid**:

1. Cluster cell boxes into rows by their y-centre, and into columns by x-centre.
2. Drop header rows (`header_rows` from the layout).
3. Map column index → field name using `layout_id`.
4. Map row index → date: `date = (year, month, row_index + 1)`. **Verify** against the handwritten date column on a few pages, because some registers skip rows or write two days on one line.
5. Crop each cell (with ~4 px padding) to `work/cells/{page_id}_r{row:02d}_c{col}.png`.
6. Write the first columns of `data/sjcet/cells.csv` (`cell_id, page_id, page_path, crop_path, bbox, date, field`).

Record which method worked per page (`tsr_method` = `glosat` / `grid` / `manual`) in `pages.csv`.

Check: for 10 pages, draw the grid with field names on the page image. Every value falls inside one box and has the right field label. Expected cell count ≈ days in month × fields.

Step 5: Decide what "blank" means

Before OCR, decide per cell whether it has ink at all: after `clean_cell.py`, if there is almost no ink (e.g. < 30 ink pixels), mark the cell `blank` and skip OCR.

Watch for these special marks, and **write down what you decide** in `NOTES.md`:

Written	Meaning	Store as
`—`, `Nil`, `0` in rain column	observed, no rain	`0`
empty	not observed	empty

Written	Meaning	Store as
`do` / `"` (ditto)	same as above	copy the value above, `qc_flag=corrected`
`Tr` (trace)	< 0.1 mm rain	`0` (note it)
struck-out values	corrected by observer	the new value

Check: blank detection agrees with your eyes on 50 random cells.

Step 6: Label training data for OCR (3–4 days, manual)

Zero-shot OCR isn't good enough, so we fine-tune it. That needs labelled examples.

1. Sample cells **stratified** across years, layouts, fields and photo vs scan. Aim for **≥ 1,500 value cells + 300 date cells** (more is better; 3,000 is great).
2. Label them in the same format as `eval/sample_cells/labels.csv`: `file,page,text,kind` where `text` is exactly what's written (e.g. `29.5`, `1.24`, `0`). Use the frontend **Review Tool** once it's ready; it's faster than a spreadsheet.
3. **Split by page, not by cell:** ~70% of pages → train, 15% → val, 15% → test. If the same page's cells are in train and test, the accuracy will be fake.

Tips: when you can't read a cell yourself, label it `?` and exclude it. The model can't learn from it either.

Check: `labels_train.csv`, `labels_val.csv`, `labels_test.csv` exist with no `page` shared between them.

Step 7: Fine-tune the OCR model (2–3 days)

Use **`microsoft/trocr-small-handwritten`**. It's fast (0.5 s/cell on CPU) and was within 2% of base zero-shot. Train on a free GPU (**Kaggle** notebooks give 30 h/week; or Google Colab), since the laptop has no CUDA.

Write `digitization/train_ocr.py` (Hugging Face `Seq2SeqTrainer` or a plain PyTorch loop):

- Inputs: `clean_cell()` output → `TrOCRProcessor` (384×384).
- Set `model.config.decoder_start_token_id`, `pad_token_id`, `eos_token_id` from the processor tokenizer.
- Hyperparameters to start: lr `5e-5`, batch 16, 15–20 epochs, AdamW, warmup 10%, fp16 on GPU, early stopping on val CER.
- **Augment** (train only): rotation ±3°, scale 0.9–1.1, small translation, Gaussian blur, brightness/contrast jitter, random erosion/dilation (pen thickness).
- Keep the digit-only restriction at inference (`ALLOWED_CHARS` in `ocr_test.py`).
- Save to `digitization/models/trocr-sjcet/` (gitignored; upload to Drive and share the link).

Evaluate on the **test** pages with `ocr_test.py` (add a `--model-path` option). Report exact match and CER per `kind` (integer / decimal / date) **before vs after**; this is a results table for the report.

Targets: ≥ 90% exact match on values. If still < 80% after two attempts:

- label more data from the worst-performing layouts/years, or
- try **PARSeq** or a per-character CNN digit classifier (segment the digits, use MNIST-pretrained weights, fine-tune), or
- for dates, skip OCR: dates are mostly implied by row position (step 4), so just check them.

Check: a results table with zero-shot vs fine-tuned accuracy on held-out pages.

Step 8: Run OCR on everything (½–1 day)

Write `digitization/run_ocr.py`: for every non-blank cell in `cells.csv`, run the fine-tuned model and fill `ocr_text` and `ocr_conf`. Confidence = mean of the per-token max softmax probabilities (`generate(..., output_scores=True, return_dict_in_generate=True)`).

It runs on CPU at ~0.5 s/cell, so ~20k cells ≈ 3 hours. Run it overnight or on Kaggle.

Check: histogram of `ocr_conf`, and how many cells fall under 0.7.

Step 9: Parse and validate (QC) (1–2 days)

Write `digitization/qc.py`:

1. **Parse** `ocr_text` → number: normalise `·` / `,` → `.`. If a decimal point is missing where one is expected (e.g. `295` for temperature), try inserting it and flag `qc_reason=decimal_inserted`.
2. **Per-value range checks** (from the frontend requirements §3.6): rain 0–40 cm, dry/wet 15–40 °C, min 15–30 °C, max 22–40 °C.
3. **Within-row consistency:** `wet ≤ dry` · `min ≤ dry ≤ max` (allow 1 °C slack) · `min < max`.
4. **Day-to-day jumps:** |Δ Tmax| > 6 °C or |Δ Tmin| > 5 °C vs neighbours → suspect.
5. **Monthly rainfall checksum:** sum of daily rain (cm) vs the monthly total on the summary PDF (`Sum/`). Record the monthly totals by hand in `work/monthly_totals.csv` (year, month, total_cm, ytd_cm). Any month that doesn't match within 0.1 cm → all rain cells in that month become `suspect` with `qc_reason=month_sum_mismatch`.
6. Low confidence: `ocr_conf < 0.7` → suspect.

Set `qc_flag` = `ok` / `suspect` / `fail` (unparseable) and `qc_reason`.

Check: % of cells per flag. Expect suspect + fail to be 5–20%.

Step 10: Human review (ongoing, several sessions)

Use the frontend **Review Tool** (it reads `cells.csv` and writes `data/sjcet/corrections.csv`):

1. Review **all** `suspect` and `fail` cells.
2. Review a **random 3–5% sample of `ok` cells**. The error rate in this sample is your **estimated residual error rate**. Report it, because examiners will ask "how accurate is the digitized data?"
3. Re-run `qc.py` after corrections (merging `corrections.csv`) and repeat until no new flags appear and every month's rain checksum matches or is explained.

Check: zero unreviewed `fail` cells. Every month checksum matches or has a note. Residual error rate measured.

Step 11: Assemble `sjcet_daily.csv` (½ day)

Write `digitization/assemble.py`:

1. Pivot cells to one row per date with the columns in data-contracts §1, applying `corrections.csv` (latest row per `cell_id` wins).
2. Convert rain **cm → mm** (× 10).
3. Insert empty rows for every missing date so the calendar is complete.

4. Compute **relative humidity** from dry bulb `T` and wet bulb `Tw` (°C), psychrometric formula for a non-ventilated (Stevenson screen) psychrometer: `es(x) = 6.112 * exp(17.67 * x / (x + 243.5)) # hPa P = 1005 hPa (station is near sea level; fine to use a constant) A = 0.000799 (per °C, Stevenson screen) e = es(Tw) - A * P * (T - Tw) RH = 100 * e / es(T) # clip to [0, 100]`
5. Set `qc_flag` per row (`corrected` if any cell in the row was corrected, `suspect` if any remained suspect).

Check: row count = days between first and last date. The file loads in pandas with the correct dtypes. RH is mostly 60–100%.

Step 12: EDA (2 days) → `digitization/eda.ipynb`

Make these plots (they go in the report and the frontend History page):

- Data coverage heatmap (year × day-of-year)
- Monthly rainfall climatology; annual totals with trend
- Tmax/Tmin annual means with trend (°C/decade)
- Monsoon onset/withdrawal per year (first/last spell of ≥ 3 rainy days ≥ 2.5 mm)
- Extreme events (e.g. 2018 Kerala floods, if covered)
- **Comparison with ERA5** at SJCET (ask the model track for the series): scatter and bias per variable. This validates both the digitization and the data the model uses.

Deliverables checklist

- [ ] `digitization/work/pages.csv`, `layouts.csv`, `monthly_totals.csv`
- [ ] `data/sjcet/cells.csv`, `corrections.csv`, **`sjcet_daily.csv`**
- [ ] Scripts: `prep_pages.py`, `assign_cells.py`, `train_ocr.py`, `run_ocr.py`, `qc.py`, `assemble.py`
- [ ] OCR results table (zero-shot vs fine-tuned, held-out pages)
- [ ] Residual error rate from the review sample
- [ ] `eda.ipynb` with the plots above
- [ ] `NOTES.md`: decisions, oddities, missing months

Suggested schedule

Days	Steps
1–3	0, 1, 2, 3 (inventory is the slow part)
4–5	4, 5
6–9	6 (labelling; start the Review Tool as soon as the frontend has it)
10–12	7
13	8, 9
14+	10, 11, 12

Hand-off early: give the model track a *partial* `sjcet_daily.csv` (e.g. the 2010–2016 scans) as soon as steps 8–11 run end-to-end on it. Don't wait for the photos.

Common pitfalls

- 1 vs 7, 3 vs 8, 5 vs 6 confusions, and **missing decimal points**. The range and consistency checks catch most of these.
- PDF is in **reverse order**, so don't assume page 1 = July 2010.

- Photo and scan of the same month → two different "truths". Use the scan.
- Splitting labels by cell instead of by page → inflated accuracy.
- Treating blank as 0 rain. They are different.